

Esame di Elettronica dei Sistemi Digitali, Reti Logiche, Elettronica per l'Innovazione del Prodotto

Prof. Roberto Passerone, Ing. Leonardo Gasparini

13 giugno 2016, Fila A

Si svolga a scelta l'intera Sezione 1 o l'intera Sezione 2, e si realizzi quindi il progetto indicato nella Sezione 3. I punti assegnati ad ogni domanda indicati sono approssimativi.

1 Circuiti logici combinatori fondamentali

- Si discuta il funzionamento di un multiplexer 2 a 1 e se ne mostri una possibile realizzazione ottenuta tramite mappe di Karnaugh.
- Si discuta il problema delle alee nel multiplexer, e come sia possibile eliminarle.
- Si discuta il funzionamento di una decodifica 2 a 4 e se ne mostri una possibile realizzazione.
- Si discuta la realizzazione di un sommatore ripple carry a 4 bit: si mostri il funzionamento del full adder e come questo venga utilizzato per realizzare il sommatore a più bit.

2 Sintesi a 2 livelli

Si descriva come avviene la semplificazione (sintesi) a 2 livelli di funzioni booleane rispondendo alle seguenti domande, facendo riferimento, ove richiesto, alla funzione booleana $f(A, B, C, D)$ in Figura 1:

- (3 pts) Qual è la differenza tra “funzione” ed “espressione”? Si faccia un esempio che identifichi chiaramente l'una e l'altra. Quali sono gli obiettivi della sintesi? Si sintetizzi quindi la funzione f in forma canonica come somma di prodotti.
- (3 pts) Quali strumenti algebrici (proprietà) abbiamo a disposizione per semplificare le espressioni? Accompagnare la risposta con un esempio applicato all'espressione ottenuta al punto precedente.
- (3 pts) Cosa si intende per implicante, implicante primo ed implicante essenziale? Se ne fornisca la definizione e si accompagni la risposta identificando gli implicant di f tramite la sua mappa di Karnaugh.
- (3 pts) Qual è il ruolo dei don't care nella semplificazione? Si evidenzino come possono essere utilizzati nell'ottimizzazione di f a partire dalla sua mappa di Karnaugh, sintetizzando la forma minima.

D	C	B	A	$f(A,B,C,D)$
0	0	0	0	-
0	0	0	1	1
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	-
0	1	1	0	1
0	1	1	1	1
1	0	0	0	-
1	0	0	1	-
1	0	1	0	0
1	0	1	1	-
1	1	0	0	0
1	1	0	1	0
1	1	1	0	0
1	1	1	1	1

Figure 1: Tabella di verità della funzione booleana da sintetizzare.

3 Sistema di gestione dinamica del traffico

Il sistema di gestione di traffico di Trento è costituito da un computer centrale che rileva il flusso di automobili nelle strade e controlla i semafori posti negli incroci sfruttando una comunicazione wireless. Ci occuperemo di gestire un singolo incrocio.

Una macchina a stati finiti MSF controlla l'accensione di due semafori posti ad un incrocio di due vie a senso unico, come mostrato in Figura 2.

Due sensori informano la MSF della presenza di macchine in arrivo nei due sensi. Il funzionamento è il seguente: la MSF mantiene il semaforo verde su di una via (per esempio via Sommarive) fin tanto che nessun veicolo arrivi dalla seconda via (via dei Valoni). Quando un veicolo arriva dalla seconda via, la MSF scambia il verde con il rosso in entrambi i sensi, purché il verde nella prima via sia rimasto acceso per un tempo minimo prefissato, scandito da un timer. Qualora il tempo minimo non sia ancora trascorso, la MSF aspetta lo scadere del timer, e quindi scambia il verde con il rosso in entrambi i sensi. A questo punto, il ruolo delle due vie si inverte, e la MSF manterrà il verde nella seconda via (via dei Valoni) fin tanto che nessun veicolo arrivi sulla prima (via Sommarive), e così via. Una macchina che arrivi su di una via che ha già il verde viene semplicemente ignorata. Per semplicità, non ci occuperemo di gestire il giallo.

Un possibile schema di implementazione è mostrato in Figura 3 e contiene la MSF, due memorie, ciascuna connessa ad un semaforo, ed un timer (realizzato come contatore). La memoria accende la luce verde quando impostata ad 1, ed accende quella rossa quando impostata a 0. Le informazioni dai sensori raggiungono la MSF in forma sincrona seriale tramite un canale wireless L . Le informazioni sono rappresentati mediante comandi di 4 bit formati da un bit di inizio comando e 3 bit di identificazione della via, così mappati:

- 1101 - Veicolo in arrivo sulla via (1);
- 1011 - Veicolo in arrivo sulla via (2).

La MSF decifra il comando ricevuto al fine di impostare il valore delle memorie che controllano i semafori. La MSF deve ignorare sequenze di bit che non corrispondono a nessun comando a causa di errori nella trasmissione, mantenendo la situazione invariata. Tuttavia, deve essere in grado di riconoscere un comando valido a seguito di bit errati. Per esempio, la ricezione della sequenza "1010..." non corrisponde a nessun comando, ma gli ultimi due bit potrebbero far parte del comando di arrivo del veicolo sulla via (2), e di conseguenza vanno tenuti in considerazione (eventualmente il comando verrà eseguito solamente dopo aver ricevuto altri 2 bit a '1', che completano il comando "1011").

Una volta ricevuto un comando, ed in base allo stato attuale dei semafori, la MSF deve verificare lo stato del timer e compiere le azioni necessarie. Il timer è dotato di un ingresso sincrono *init* che lo re-imposta ad una situazione iniziale, in cui l'uscita di Terminal Count (TC) è bassa. Una volta rilasciato il segnale di *init*, il timer è abilitato al conteggio. Dopo un tempo prefissato (per esempio, 10 colpi di clock, fondo scala 9), l'uscita TC viene abilitata e resta alta fino all'*init* successivo. Le memorie sono sincrone, e possono essere impostate a '1'/'0' mediante i rispettivi segnali *setM1/2* e *clearM1/2*. Quando entrambi i segnali sono a 0, la memoria non cambia di stato.

Note:

- Si può assumere una distanza di 3 cicli di clock tra un comando valido e quello successivo.
- Mentre si attende il timer, non è necessario verificare la presenza di ulteriori comandi, essendo questi ininfluenti.

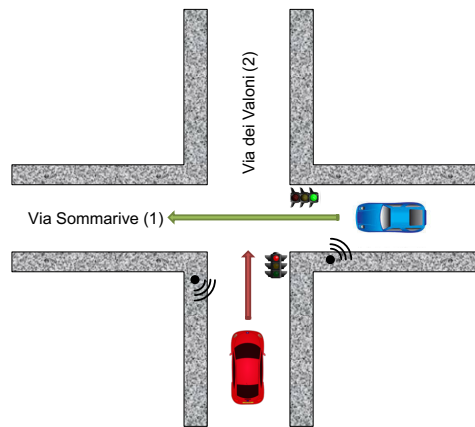


Figure 2: Schema dell'incrocio

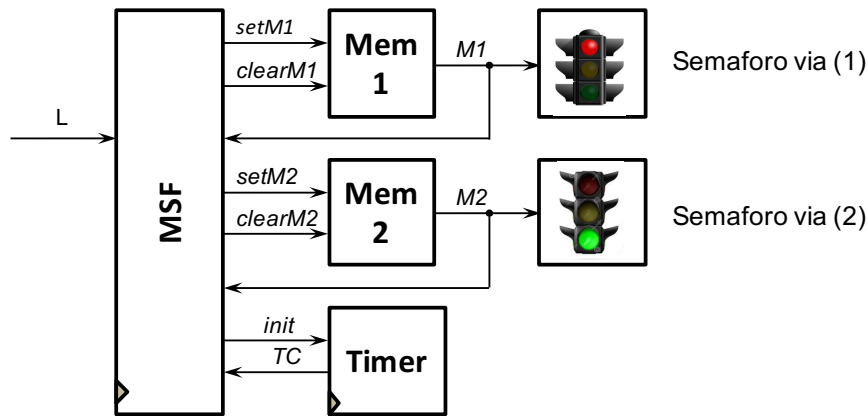


Figure 3: Schema del circuito

- La MSF riceve in ingresso il valore delle due memorie $M1$ ed $M2$. Questo dato può essere utilizzato per semplificare il progetto della macchina a stati. Soluzioni alternative che non ne facciano uso sono comunque considerate accettabili.
- Si assuma che all'accensione il timer sia già al fondo scala, e che $M2 = 1$ ed $M1 = 0$.

Si richiede di:

- (9 pts) Progettare la macchina a stati finiti mostrando e commentando a parole il diagramma degli stati.
- (4 pts) Completare il diagramma temporale mostrato in Figura 4, supponendo che nel primo colpo di clock sia verde il semaforo 2 ($M2 = 1$) e che TC sia già alto.
- (5 pts) Si sviluppi il codice VHDL della memoria che controlla il semaforo, facendo in modo che il segnale di *clear* sia prioritario rispetto a quello di *set*.

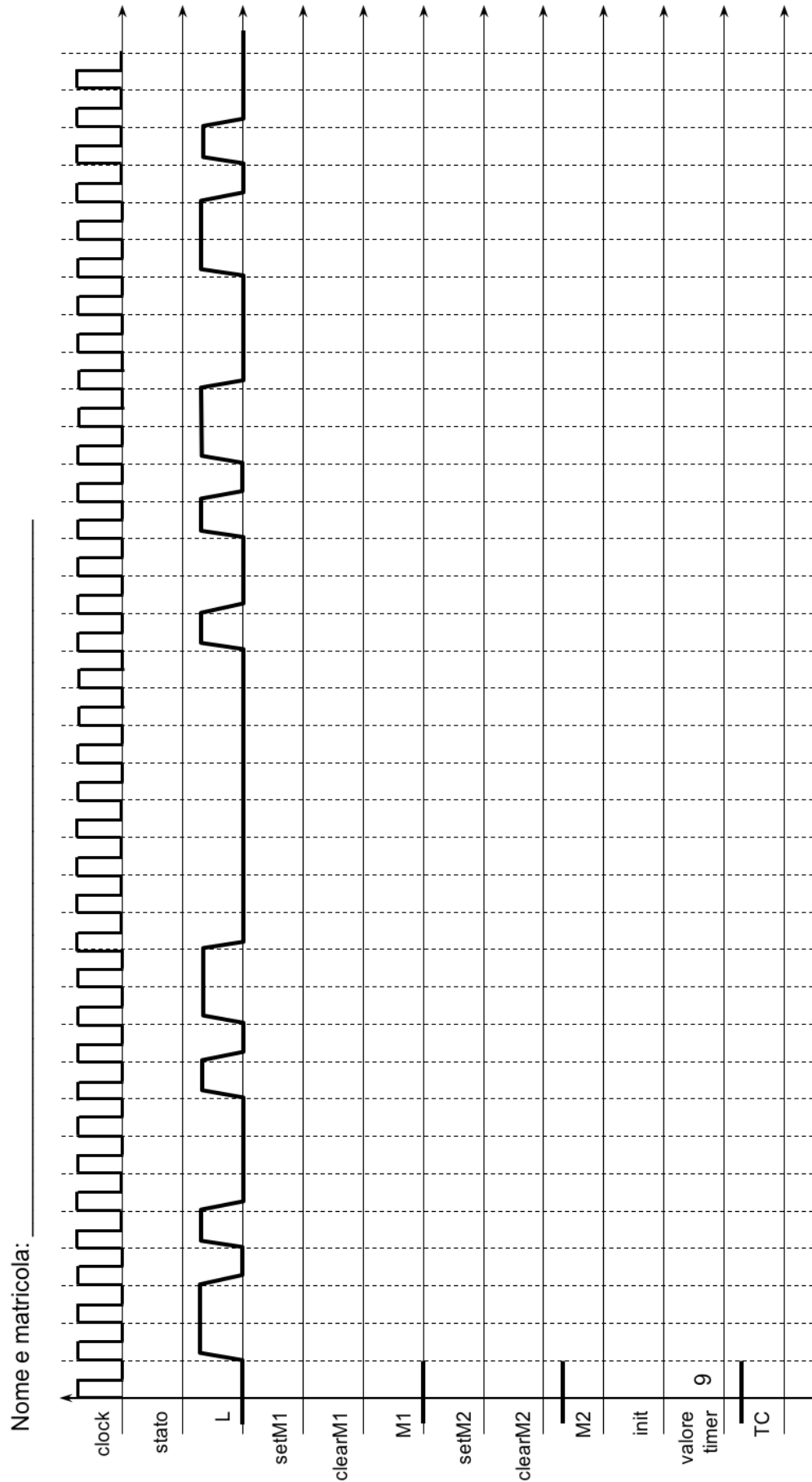


Figure 4: Diagramma temporale